

MLA0303-Reinforcement Learning

CO-2 AT-2

Application - Based Questions

Name: S. Divya

Reg No: 192425163

1. Ride-Sharing: Design and Develop an RL Framework

Application Requirement: The ride-sharing company needs to match passengers with suitable drivers quickly. The main application objective is to minimize passenger waiting time while maintaining good driver utilization and service quality.

RL Model / Required Concept

Application Flow

Passenger requests + driver/GPS data → **State** → **Match driver** → **Ride outcome** → **Reward** → **Policy update** → **Next match**

Implementation / Tools: Use GPS/trip history, traffic APIs, demand heat maps, dispatch databases and simulation before deployment.

Accuracy & Problem-Solving: Evaluate average waiting time, cancellation rate, successful-match rate, driver utilization and pickup distance. Hard constraints must prevent duplicate assignments and infeasible matches.

Conclusion: Therefore, the RL framework directly addresses the application requirement by learning an adaptive matching policy that reduces waiting time and improves service quality.

RL Framework Details

State: Driver locations and availability, passenger pickup locations, current demand, traffic, estimated waiting time, destination zone, vehicle type and time of day.

Action: Assign an available driver to a passenger, select another feasible driver, reposition an idle driver, or reassign a pending request.

Reward: Positive reward for successful and quick matches and balanced driver utilization. Negative reward for passenger waiting time, long pickup distance, cancellations and idle time.

Policy: $\pi(s)$ selects the matching/repositioning action with the highest expected long-term service reward.

Framework: At each decision point the agent observes the platform state, chooses a feasible action, receives the service outcome and reward, and updates its policy/value estimates.

2. E-Learning: Exploration and Exploitation

Application Requirement: The e-learning platform must recommend personalized study material. The application specifically requires a balance between recommending material already known to work for the learner and trying new content that may improve learning. This is the exploration–exploitation trade-off.

RL Model / Required Concept

Application Flow

Learner state → **Explore/Exploit decision** → **Study material** → **Learning feedback** → **Reward** → **Update recommendation policy**

Implementation / Tools: Exploration is useful for discovering new interests and weak topics; exploitation gives reliable recommendations. The exploration rate should reduce when the system becomes confident about the learner.

Accuracy & Problem-Solving: Performance can be measured using mastery improvement, completion, retention, satisfaction and recommendation relevance. Exploration should never dominate the learner experience.

Conclusion: The correct RL strategy is therefore a controlled balance: exploitation provides reliable personalized study, while exploration discovers better learning opportunities.

RL Framework Details

State: Learner history, topic mastery, quiz scores, recent mistakes, preferred difficulty, completed lessons and current learning goal.

Action: Recommend a familiar resource, introduce a new resource, select difficulty, or recommend a quiz/revision item.

Exploitation: Use known high-performing materials because they have already produced good learning outcomes for this learner.

Exploration: Try new topics, formats or difficulty levels to discover resources that may be better than the current choice.

Balance: Use controlled exploration, such as ϵ -greedy or contextual bandits: most decisions exploit the best-known content while a small portion explores alternatives.

Reward: Reward mastery improvement, quiz performance and meaningful completion; penalize irrelevant content and abandonment.

3. Robotic Vacuum: Policy and Value Function

Application Requirement: The robotic vacuum must clean efficiently while avoiding obstacles. The application requires the robot to use a policy for action selection and a value function to estimate how useful future states are. RL allows these components to improve through repeated interaction.

RL Model / Required Concept

Application Flow

Sensor observation → **State** → **Policy chooses movement** → **Environment changes** → **Reward** → **Value update** → **Improved policy**

Implementation / Tools: Training can use robot simulation, LiDAR/camera/distance sensors, mapping software and battery sensors. Simulation allows learning without repeated physical collisions.

Accuracy & Problem-Solving: Track floor coverage, cleaning time, collision count, repeated coverage and battery efficiency. Safety rules must override dangerous learned actions.

Conclusion: Thus, the policy chooses actions while the value function guides the policy toward states with better long-term cleaning performance.

RL Framework Details

State: Robot position/orientation, obstacle distances, cleaned and uncleaned regions, dirt detection and battery level.

Action: Move forward, turn left/right, move backward, slow down, clean a region, or return to the charging station.

Reward: Reward new floor coverage and efficient cleaning. Penalize collisions, repeated coverage, excessive movement and battery depletion.

Policy: The policy $\pi(a|s)$ maps the current sensor state to the preferred safe movement action.

Value Function: $V(s)$ estimates expected future cumulative reward from state s . A state that can lead to large safe cleaning coverage has a high value.

Improvement: After each movement, the robot observes the next state and reward and updates $V(s)$ and/or the policy, making useful actions more likely over time.

4. Cryptocurrency Trading: Reward Function Challenges

Application Requirement: The cryptocurrency agent chooses buy, sell or hold actions. The main application issue is designing a reward that represents genuine trading performance. A badly designed reward can make the agent optimize an incorrect objective and appear successful during training while performing poorly in realistic trading.

RL Model / Required Concept

Application Flow

Market state → **Buy/Sell/Hold** → **Portfolio change** → **Reward** → **Policy update** → **Next trading decision**

Implementation / Tools: The reward should represent the complete objective rather than one metric. Risk penalties and transaction costs make the reward closer to real-world performance.

Accuracy & Problem-Solving: Use walk-forward validation and unseen market periods because cryptocurrency markets are non-stationary. Position and loss limits should be hard constraints.

Conclusion: Therefore, reward design is critical: the agent learns exactly what the reward encourages, so poor reward design directly produces poor or risky trading behavior.

RL Framework Details

State: Price/returns, volume, volatility, technical indicators, current holdings, cash balance and recent market conditions.

Action: Buy/increase position, sell/decrease position, or hold, with position-size limits.

Reward Design: Use risk-adjusted portfolio return and subtract transaction costs, volatility/drawdown and excessive turnover.

Poor Reward Problem: Rewarding only immediate profit can encourage overtrading, extreme risk or exploitation of short-term noise.

Reward Hacking: The agent may maximize the numerical reward without achieving the intended business objective, for example by repeatedly trading when the reward ignores transaction costs.

Validation: Backtest on unseen periods and evaluate return, Sharpe ratio, drawdown, volatility, turnover and transaction costs.

5. Smart Grid: Create an RL Model

Application Requirement: The smart grid must distribute electricity while balancing demand, renewable generation, battery storage and grid constraints. The requested RL model is an MDP in which the energy-management agent makes repeated decisions.

RL Model / Required Concept

Application Flow

Observe grid state → Select energy action → Grid response → Cost/reliability reward → Update policy/value → Next interval

Implementation / Tools: Use smart-meter data, renewable sensors, battery-management systems, weather forecasts and grid simulation/digital twins.

Accuracy & Problem-Solving: Evaluate energy cost, renewable utilization, unmet load, peak demand and battery degradation. Physical grid and battery limits must be hard constraints.

Conclusion: This RL model is appropriate because electricity management is sequential, dynamic and uncertain, allowing the policy to adapt to changing demand and renewable supply.

RL Framework Details

State Space: Electricity demand, solar/wind generation, battery state of charge, electricity price, time, weather forecast and grid condition.

Action Space: Charge/discharge battery, use renewable power, import/export electricity, or shift approved flexible loads.

Reward Mechanism: Reward reliable supply and renewable utilization; penalize energy cost, unmet demand, battery degradation, peak demand and unnecessary curtailment.

Policy: $\pi(a|s)$ selects the energy-management action that maximizes long-term reward under operating constraints.

Transition: After an action, demand, renewable generation and battery state change, producing the next state.

Objective: Minimize cost and maximize reliability/renewable utilization over the complete operating horizon.

6. Healthcare Monitoring: States, Actions, Rewards and Learning

Application Requirement: A healthcare monitoring system can use RL as a clinical decision-support framework for adapting treatment recommendations over time. Because medication is safety-critical, RL should operate only within approved clinical protocols and clinician supervision; it should not independently prescribe unsafe doses.

RL Model / Required Concept

Application Flow

Patient data → **Validated state** → **Approved action/clinician review** → **Outcome** → **Reward** → **Value/policy update**

Implementation / Tools: Use validated clinical datasets, monitoring devices, simulation and clinical decision-support dashboards. Evaluation must prioritize safety and clinical validity.

Accuracy & Problem-Solving: Measure adverse-event rate, target achievement, robustness and calibration rather than only numerical reward.

Conclusion: RL can model long-term treatment effects, but the application must remain clinician-supervised and safety-constrained.

RL Framework Details

State: Validated patient measurements, symptoms, recent treatment history, adherence, relevant laboratory results and patient context.

Action: Continue approved treatment, make an approved treatment adjustment, or request clinician review. Exact medication decisions remain under authorized clinical protocols.

Reward: Reward movement toward approved clinical targets and stable outcomes; penalize adverse events, unsafe deviations and unnecessary interventions.

Policy: The policy maps the patient state to an allowed recommendation or clinician-review action.

Learning: Outcomes from longitudinal data or validated simulation update value estimates/policy so that safer, more effective permitted decisions become more likely.

Safety: Clinical rules, clinician approval, privacy, audit logs and emergency fallback have priority over reward maximization.

7. Drone Delivery: Exploration and Exploitation

Application Requirement: A delivery drone must find efficient routes while avoiding obstacles, restricted airspace and unsafe weather. Exploration and exploitation affect how the navigation policy learns route choices.

RL Model / Required Concept

Application Flow

Drone sensors + map/weather → **State** → **Explore or exploit route** → **Flight outcome** → **Reward** → **Policy update**

Implementation / Tools: Use drone simulators, maps, GPS/IMU, obstacle sensors and weather data. Simulation is the safest environment for exploration.

Accuracy & Problem-Solving: Compare delivery time, route length, energy use, successful delivery rate and safety violations.

Conclusion: RL improves route optimization by discovering alternatives during exploration and using proven safe routes during exploitation.

RL Framework Details

State: GPS position, altitude, battery, destination, weather/wind, obstacles, no-fly zones and estimated travel time.

Action: Choose a route segment, change direction/altitude/speed, hover, reroute or return to a safe location.

Reward: Reward progress, successful delivery and energy efficiency; penalize collisions, restricted-zone entry, delay and excessive energy use.

Exploration: Try alternative safe routes in simulation to discover shorter or more energy-efficient paths.

Exploitation: Use validated high-performing routes when the policy has sufficient confidence, especially during real delivery.

Balance: Explore mainly in simulation/controlled testing and exploit validated policies in real flights because unsafe exploration has high cost.

8. Manufacturing Robot: Policy and Value Function

Application Requirement: The manufacturing robot must assemble products accurately and efficiently. The application asks for a policy and value function: the policy selects assembly actions, while the value function estimates the long-term usefulness of each assembly state.

RL Model / Required Concept

Application Flow

Robot sensors → **Assembly state** → **Policy action** → **Assembly outcome** → **Reward** → **Value update** → **Better policy**

Implementation / Tools: Use robot simulation/digital twins, force sensors, cameras, robot-controller logs and quality-inspection data.

Accuracy & Problem-Solving: Evaluate cycle time, defect rate, collision count, energy use and successful completion rate.

Conclusion: Policy and value functions together allow the robot to learn not just immediate actions but action sequences that produce better long-term assembly performance.

RL Framework Details

State: Robot joint positions, part position, assembly stage, gripper status, force/torque readings and alignment condition.

Action: Move, grasp, release, adjust position, perform an approved assembly motion, slow down, retry or request inspection.

Reward: Reward successful defect-free assembly, precision and low cycle time; penalize collisions, excessive force, failed assembly and rework.

Policy: $\pi(a|s)$ chooses the next assembly action for the current state.

Value Function: $V(s)$ estimates expected cumulative future reward. States leading to successful and efficient completion receive higher values.

Learning: Observed successes/failures update value estimates, allowing the policy to prefer action sequences with higher long-term efficiency.

9. Online Advertising: Reward Design Challenges

Application Requirement: An online advertising agent selects advertisements for users. The application objective is not simply to maximize clicks; a useful system should also consider conversions, revenue and long-term user experience. Reward design determines which advertising behavior the RL agent learns.

RL Model / Required Concept

Application Flow

User/context → **Candidate ads** → **Select ad** → **User response** → **Reward** → **Policy update** → **Next ad**

Implementation / Tools: Use privacy-compliant ad logs, campaign data, conversion events and A/B testing.

Accuracy & Problem-Solving: Evaluate CTR together with conversion rate, revenue, retention, negative feedback and user experience. This prevents a single metric from dominating the objective.

Conclusion: Correct reward design aligns the learned policy with the actual business and user objective instead of allowing short-term CTR optimization to harm the platform.

RL Framework Details

State: User/context features, page context, previous ad interactions, device, time, campaign information and candidate advertisements.

Action: Select an approved advertisement or ranked set of advertisements.

Reward: Combine meaningful clicks/conversions and revenue with penalties for poor user experience, excessive repetition and policy violations.

Challenge: If reward equals CTR only, the agent may prefer clickbait that generates clicks without meaningful conversion or satisfaction.

Impact of Poor Reward: The system may increase short-term CTR while reducing conversion quality, retention and user trust.

Improved Design: Use multi-objective or constrained rewards and validate using offline testing and controlled experiments.

10. Smart Irrigation: Create an RL Framework

Application Requirement: The smart irrigation system must maintain suitable soil moisture while reducing water waste. Weather, rainfall and crop requirements change over time, so RL can learn a sequential irrigation policy.

RL Model / Required Concept

Application Flow

Soil + weather + crop state → **Irrigation action** → **Soil moisture change** → **Reward** → **Policy update** → **Next decision**

Implementation / Tools: Use soil-moisture sensors, weather stations/APIs, flow meters, water-level sensors, IoT controllers and field records.

Accuracy & Problem-Solving: Evaluate water saved, soil-moisture stability, irrigation frequency, pump energy and crop-health/yield indicators. Extreme actions should be prevented by safety/agronomic limits.

Conclusion: RL provides adaptive irrigation because it learns from changing soil and weather conditions while optimizing water use over multiple decisions.

RL Framework Details

State Space: Soil moisture, rainfall, weather forecast, temperature, humidity, crop growth stage, soil type, water level and irrigation history.

Action Space: No irrigation, short/medium/long irrigation, or select an approved irrigation zone/flow setting.

Reward Function: Reward appropriate soil moisture and crop conditions; penalize water waste, over-irrigation, under-irrigation risk and unnecessary pump energy.

Policy: $\pi(a|s)$ maps current soil/weather/crop conditions to the irrigation action expected to provide the best long-term outcome.

Transition: After irrigation and weather changes, soil moisture and water availability change, creating the next state.

Learning: The system observes the result, receives the reward and updates value/policy estimates for future irrigation decisions.